Plain-English Guide to the VinePPO Testbed: Environments, Transformer Critic, Batched FBI, and Staged FBI

Project Overview & Visual Walkthrough

November 9, 2026

1 What This Project Is Doing (Big Picture)

When training a Large Language Model (LLM) to solve multi-step math or reasoning problems using Reinforcement Learning (RL), we usually only get a reward at the very end of the solution: $R = 1$ if the final answer is right, and $R = 0$ if it is wrong.

To train the model effectively with PPO (Proximal Policy Optimization), we need a **Critic** (value function $V_\phi(s_t)$) that can look at an incomplete reasoning chain at step t and estimate: *"What is the probability that the model will eventually reach a correct final answer from this point?"* Once we know the value of every step, we can tell whether a specific reasoning step a_t improved or hurt our chances of success via the step-level advantage:

$$\hat{A}(s_t, a_t) = r_t + \gamma V(s_{t+1}) - V(s_t).$$

Rather than running expensive LLM decoding experiments to study how best to train this critic, this project builds a fast, exact, pure-JAX synthetic testbed where we **know the exact ground-truth value $V^*(s)$ of every state** via dynamic programming, allowing us to directly measure how fast and accurately different RL value-learning algorithms converge.

2 The Environments (MRPs / MDPs)

The codebase models multi-step reasoning using two synthetic Markov environments where observations are sequences of tokens fed into a Transformer, and rewards are sparse and binary ($R \in \{0, 1\}$ at the end of the episode):

1. **Binary Tree MRP (`envs/tree_mrp.py`):**
   - **Structure:** A complete binary tree of depth D (e.g., $D = 4$ or $D = 7$) with $2^{D+1} - 1$ states. Starting at the root s_0, at each step the trajectory branches Left (`0`) or Right (`1`).
   - **Rewards & Dynamics:** Only a small cluster of leaf nodes at depth D are "correct solutions" yielding $R = 1$; all other leaves yield $R = 0$. At any internal node where one child subtree contains a correct leaf and the other is a dead end, the base policy picks the correct branch with probability p (e.g., $p = 0.8$) and blunders into the dead-end subtree with probability $1 - p$.
   - **Observation:** The sequence of binary choices taken so far, padded to length D (e.g., `[1, 0, 1, PAD, PAD]`).
2. **Procedural Reasoning DAG (`envs/reasoning_dag_env.py`):**

- **Why a DAG instead of a Tree?** Real reasoning is not a strict tree: different valid chains of thought can reconverge to the same intermediate milestone, and if a model makes a mistake (derails), it can verify/self-correct and get back on track.
- **Hidden Markov State:** Each problem requires completing M sequential **milestones** within T maximum steps ($T > M$). The true underlying state is tracked by the tuple (`prompt_idx`, step t, milestone $m \in \{0..M\}$, `is_derailed` $d \in \{0, 1\}$).
- **Transitions:** At each step, the base policy chooses from $|A|$ discrete action tokens. When on-track ($d = 0$), picking a valid advance token increments the milestone ($m \to m + 1$, probability $p_{\text{advance}} = 0.65$), picking a neutral token keeps ($m, d = 0$), and picking a blunder token derails the trajectory ($d \to 1$, probability $p_{\text{blunder}} = 0.15$). Once derailed ($d = 1$), normal actions fail to advance m; the trajectory can only return to $d = 0$ by emitting a special **self-correction/verification action** (probability $p_{\text{recover}} = 0.35$).
- **What the Critic Sees:** Crucially, the network **never** sees the privileged (m, d) variables. It only sees the raw sequence of emitted token IDs: `[BOS, PROMPT_ID, SEP,` a_0, a_1, `...,` a_{t-1}, `PAD, ...]`. It must infer which milestone has been reached and whether the chain of thought is currently derailed purely from the token history.

In both environments, an exact backward-induction solver (`solve_backward_induction()`) precomputes the true value $V^*(s)$ and true action advantages $A^*(s, a)$ for every state so we can evaluate the critic's Mean Squared Value Error (MSVE) and Advantage Cosine Similarity at every training step.

3 The Network (`transformer.py`)

The critic $V_\phi(s_t)$ is a modern, miniature LLaMA-style causal Transformer (`TransformerCritic`):

1. **Token Embedding:** Maps the discrete observation sequence (and an optional prepended `BOS` token) into d_{model}-dimensional vectors (default $d_{\text{model}} = 64$).
2. **Causal Transformer Blocks ($N = 2$ layers, $H = 4$ heads):** Each layer applies Pre-RMSNorm, Multi-Head Causal Self-Attention with Rotary Position Embeddings (RoPE), and a SwiGLU feed-forward network.
3. **Last-Valid-Token Pooling:** Extracts the hidden state at the last non-padding token position (which summarizes the entire reasoning prefix up to step t).
4. **MLP & Value Head:** Passes that pooled vector through a 2-layer SiLU MLP with RMSNorm and a zero-initialized linear projection head to output a scalar value prediction $V_\phi(s_t) \in \mathbb{R}$.

4 The Value-Learning Algorithms

Every algorithm starts an outer training update by rolling out a batch of $B = 64$ primary "base trajectories" (chains) $\tau = (s_0, a_0, s_1, \ldots, s_T)$, and differs in how it constructs and fits the regression targets $y(s_t)$ for the states along that chain.

4.1 Algorithm 1: Standard Monte Carlo

- **How it works:** Generates only the single base trajectory ($K = 1$). Every intermediate state s_t along that trajectory is trained to predict the final binary reward $R(s_T) \in \{0, 1\}$ of that single rollout ($y_t^{\text{MC}} = R(s_T)$ for all $t \in \{0, \ldots, T\}$).
- **Trade-off:** Cheapest rollout cost ($\mathcal{O}(T)$ transitions per trajectory), but highest variance: the same intermediate state s_t receives target 1 on one trajectory and 0 on another.

4.2 Algorithm 2: Vine-Target (Full-Horizon Branching)

- **How it works:** Inspired by VinePPO, at *every* visited state s_t along the base trajectory, the environment state is cloned K times (e.g., $K = 4$) and rolled out **all the way to the end of the episode** ($t \to T$). The average terminal reward $\hat{V}_{\mathrm{MC}}(s_t) = \frac{1}{K}\sum_{k=1}^{K} R(\tau_{t\to T}^{(k)})$ is used as a low-variance target.
- **Optional Precision-Weighted Shrinkage:** Because $\hat{V}_{\mathrm{MC}}(s_t)$ is an average of K Bernoulli trials, its sampling variance is $\hat{\sigma}^2_{\mathrm{MC}}(s_t) = \frac{\hat{V}_{\mathrm{MC}}(1-\hat{V}_{\mathrm{MC}})}{K-1}$. We can optionally blend $\hat{V}_{\mathrm{MC}}(s_t)$ with the current critic prediction $V_\phi(s_t)$ using inverse-variance weights when the critic is already accurate.
- **Trade-off:** Unbiased Monte Carlo targets (zero bootstrapping drift), but requires $\mathcal{O}(K \cdot T^2)$ environment transitions per trajectory because every intermediate branch must be rolled out all the way to horizon T.

The Core Idea of Fitted Backward Induction (FBI) — 1-Step Branching: Instead of rolling out K branches all the way to the end of the episode (T), **Fitted Backward Induction (FBI)** rolls out K parallel branches for **only 1 step** from each state s_t on the primary chain:

$$s_t \xrightarrow{a_t^{(k)}\sim\pi} \big(r_t^{(k)},\ s_{t+1}^{(k)}\big), \qquad k = 1, \ldots, K,$$

reducing the total rollout cost from $\mathcal{O}(K\cdot T^2)$ to $\mathcal{O}(K\cdot T)$. Given critic parameters θ, we approximate the expected 1-step Bellman backup at s_t by averaging over those K children:

$$\hat{V}_{\mathrm{branch}}(s_t;\theta) \ = \ \frac{1}{K}\sum_{k=1}^{K}\Big(r_t^{(k)} + \gamma\, V_\theta\big(s_{t+1}^{(k)}\big)\Big),$$

where $V_\theta(s_T^{(k)}) \equiv 0$ after a terminal step (so at the penultimate step $t = T-1$, $\hat{V}_{\mathrm{branch}}(s_{T-1}) = \frac{1}{K}\sum_{k=1}^{K} R(s_T^{(k)})$ is an **exact** average of terminal rewards with no critic bootstrap). To optionally blend this 1-step branched backup with longer-horizon returns along the primary chain via $\lambda \in [0,1]$, we define the branched TD residual $\delta_t^{\mathrm{branch}}(\theta) = \hat{V}_{\mathrm{branch}}(s_t;\theta) - V_\theta(s_t)$ and the general λ-return target:

$$y_t^{(\lambda)}(\theta) \ = \ \hat{V}_{\mathrm{branch}}(s_t;\theta) \ + \ \gamma\lambda\, \hat{A}_{t+1}^{(\lambda)}(\theta), \qquad \text{where} \quad \hat{A}_t^{(\lambda)}(\theta) = \delta_t^{\mathrm{branch}}(\theta) + \gamma\lambda\, \hat{A}_{t+1}^{(\lambda)}(\theta) \quad (\hat{A}_T^{(\lambda)} = 0).$$

When $\lambda = 0$, the target simplifies to pure 1-step backward induction: $y_t^{(0)}(\theta) = \hat{V}_{\mathrm{branch}}(s_t;\theta)$. Depending on *which critic parameters* θ are plugged into $y_t^{(\lambda)}(\theta)$ and *in what order* the critic is trained, we obtain two distinct algorithms: **Batched FBI** (Algorithm 3) and **Staged FBI** (Algorithm 4).

4.3 Algorithm 3: Batched FBI — $\mathbf{FBI_{batched}}(\lambda)$

Batched FBI is the simpler, synchronous algorithm (Figure 1). Within each outer training iteration, it computes all targets across the entire trajectory first using the fixed pre-update critic parameters ϕ_{old}, and then fits the network on all timesteps simultaneously:

1. **Step 1 (Rollout Chain + K 1-Step Branches):** Roll out the primary chain $\tau = (s_0, s_1, \ldots, s_T)$ and, at each state s_t ($t \in \{0, \ldots, T-1\}$), sample K independent 1-step transitions $s_t \to s_{t+1}^{(1..K)}$.
2. **Step 2 (Compute All Targets Using Fixed ϕ_{old}):** Freeze the current critic weights $\theta = \phi_{\mathrm{old}}$ and compute the target y_t for every step $t \in \{0, \ldots, T-1\}$:

$$y_t \ = \ y_t^{(\lambda)}(\phi_{\mathrm{old}}) \ = \ \hat{V}_{\mathrm{branch}}(s_t;\phi_{\mathrm{old}}) + \gamma\lambda\, \hat{A}_{t+1}^{(\lambda)}(\phi_{\mathrm{old}}), \qquad \forall\, t \in \{0, \ldots, T-1\}.$$

3. **Step 3 (Fit All Timesteps Simultaneously):** Collect the full-trajectory dataset $\mathcal{D}_{\mathrm{all}} = \big\{(s_t, y_t)\big\}_{t=0}^{T-1}$ and update ϕ by minimizing the mean squared error over all timesteps together for E passes (epochs):

$$\min_\phi \ \mathcal{L}_{\mathrm{batched}}(\phi) \ = \ \frac{1}{T}\sum_{t=0}^{T-1}\Big(V_\phi(s_t) - y_t\Big)^2.$$

4.4 Algorithm 4: Staged FBI — $\mathbf{FBI_{staged}}(\lambda)$

Why Staged FBI? In Batched FBI with $\lambda = 0$, the target at step t is $y_t = \frac{1}{K}\sum_{k=1}^{K}\big(r_t^{(k)} + \gamma V_{\phi_{\text{old}}}(s_{t+1}^{(k)})\big)$. Early in training, before V_ϕ has learned the values at step $t+1$, querying the pre-update network $V_{\phi_{\text{old}}}(s_{t+1}^{(k)})$ yields uncalibrated predictions, so it takes at least T separate outer rollout iterations for terminal reward information at step T to propagate all the way back to s_0.

Staged FBI eliminates this lag by solving the value function in **reverse backward-induction order ($t = T-1 \to 0$)** within a single iteration, updating the critic parameters $\phi^{(T-1)} \to \phi^{(T-2)} \to \cdots \to \phi^{(0)}$ at step $t+1$ *before* querying them to construct the target at step t (Figure 2):

1. **Step 1 (Rollout Chain + K 1-Step Branches):** Identical to Batched FBI: roll out the primary chain $\tau = (s_0, \ldots, s_T)$ and fork K 1-step branches $s_{t+1}^{(1..K)}$ at each state s_t.
2. **Step 2A (Base Case at the End of the Chain, $t = T-1$):** At the penultimate step $t = T-1$, all K 1-step branches land on terminal states $s_T^{(1..K)}$ where exact terminal rewards $R(s_T^{(k)}) \in \{0, 1\}$ are observed directly from the environment. Thus the target $y_{T-1} = \frac{1}{K}\sum_{k=1}^{K} R(s_T^{(k)})$ (and $y_T = R(s_T)$) requires no critic bootstrapping. Initialize the active training set $\mathcal{D}_{T-1} = \big\{(s_{T-1}, y_{T-1}),\ (s_T, y_T)\big\}$ and run M gradient steps on $\mathcal{D}_{T-1}$ to obtain calibrated terminal-stage parameters $\phi^{(T-1)}$.
3. **Step 2B (Backward Induction Loop for $t = T-2$ down to 0):** Proceeding backward one step at a time from $t = T-2$ down to 0:
   - **Compute Target at Step t Using Just-Fitted Parameters $\phi^{(t+1)}$:** Query the critic $V_{\phi^{(t+1)}}$ (which was just trained on step $t+1$) on the K 1-step children of s_t:

$$y_t \;=\; y_t^{(\lambda)}\big(\phi^{(t+1)}\big) \;=\; \frac{1}{K}\sum_{k=1}^{K}\Big(r_t^{(k)} + \gamma\, V_{\phi^{(t+1)}}\big(s_{t+1}^{(k)}\big)\Big) \;+\; \gamma\lambda\, \hat{A}_{t+1}^{(\lambda)}.$$

   - **Add (s_t, y_t) to Cumulative Active Set $\mathcal{D}_t$:** Because V_ϕ is a single shared neural network, training *only* on step t would overwrite what the network just learned at deeper steps $t+1 \ldots T$. We therefore keep all resolved steps $[t \ldots T]$ in the active set: $\mathcal{D}_t = \mathcal{D}_{t+1} \cup \big\{(s_t, y_t)\big\} = \big\{(s_\tau, y_\tau)\big\}_{\tau=t}^{T}$.
   - **Run M Gradient Steps to Obtain $\phi^{(t)}$:** Update the critic from $\phi^{(t+1)}$ to $\phi^{(t)}$ by taking M gradient steps on the cumulative squared loss $\mathcal{L}_t(\phi) = \frac{1}{|\mathcal{D}_t|}\sum_{\tau=t}^{T}\big(V_\phi(s_\tau) - y_\tau\big)^2$, then step backward ($t \leftarrow t-1$) until the root $t = 0$ is fitted.

4.5 Head-to-Head Summary: Batched FBI vs. Staged FBI

Property	Algorithm 3: Batched FBI	Algorithm 4: Staged FBI
Rollout Data	Primary chain + K 1-step branches per state	Primary chain + K 1-step branches per state
Target Formula at Step t	$y_t = y_t^{(\lambda)}(\phi_{\text{old}})$ (uses fixed pre-update weights)	$y_t = y_t^{(\lambda)}(\phi^{(t+1)})$ (uses weights just fitted at $t+1$)
Training Objective	$\frac{1}{T}\sum_{t=0}^{T-1}(V_\phi(s_t) - y_t)^2$ across all t at once	Sequential stages $t = T-1 \to 0$ on resolved set $\mathcal{D}_t = [t \ldots T]$
Weighting Across Steps	**Uniform:** Every step t receives equal weight	**Bottom-heavy:** Deeper steps $\tau \approx T$ stay in $\mathcal{D}_t$ across more stages
Key Advantage	**Lower final asymptotic error** (uniform weighting)	**Faster early learning** (rewards propagate $T \to 0$ in 1 iteration)

Table 1: **Mathematical comparison of Batched FBI and Staged FBI.**

5 Visual Diagram 1: Batched FBI ($\mathbf{FBI_{batched}}(\lambda)$)

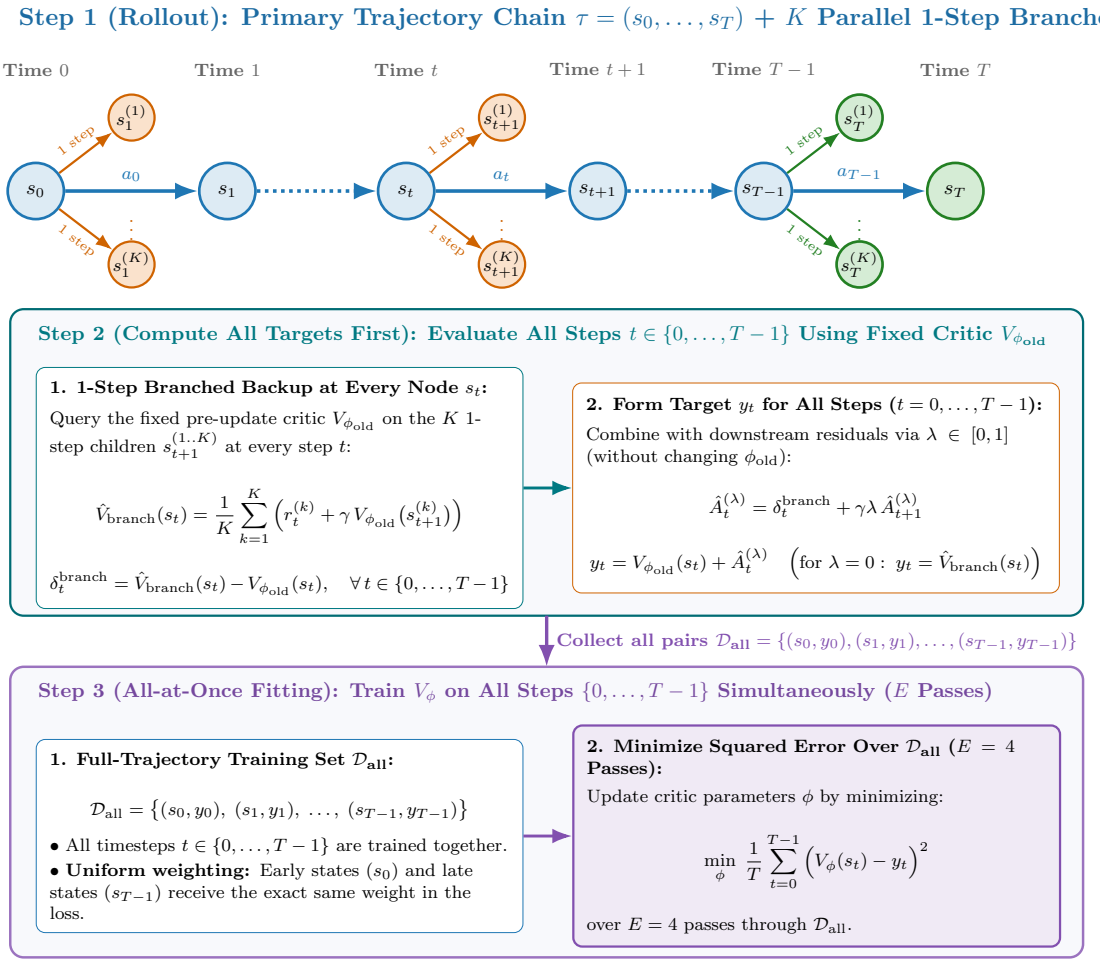


Figure 1: **Algorithm 3: Batched FBI ($\mathbf{FBI_{batched}}(\lambda)$). Top (Step 1):** Roll out the primary chain $\tau = (s_0, \ldots, s_T)$ (blue/green circles) and fork K parallel 1-step branches $s_{t+1}^{(1..K)}$ (orange/green circles) from each visited state s_t. **Middle (Step 2):** Using the fixed pre-update critic $V_{\phi_{\text{old}}}$, compute the targets $y_0, \ldots, y_{T-1}$ for all steps first. **Bottom (Step 3):** Train V_ϕ on all steps $\{0, \ldots, T-1\}$ simultaneously for $E = 4$ passes.

6 Visual Diagram 2: Staged FBI ($\mathbf{FBI_{staged}}(\lambda)$)

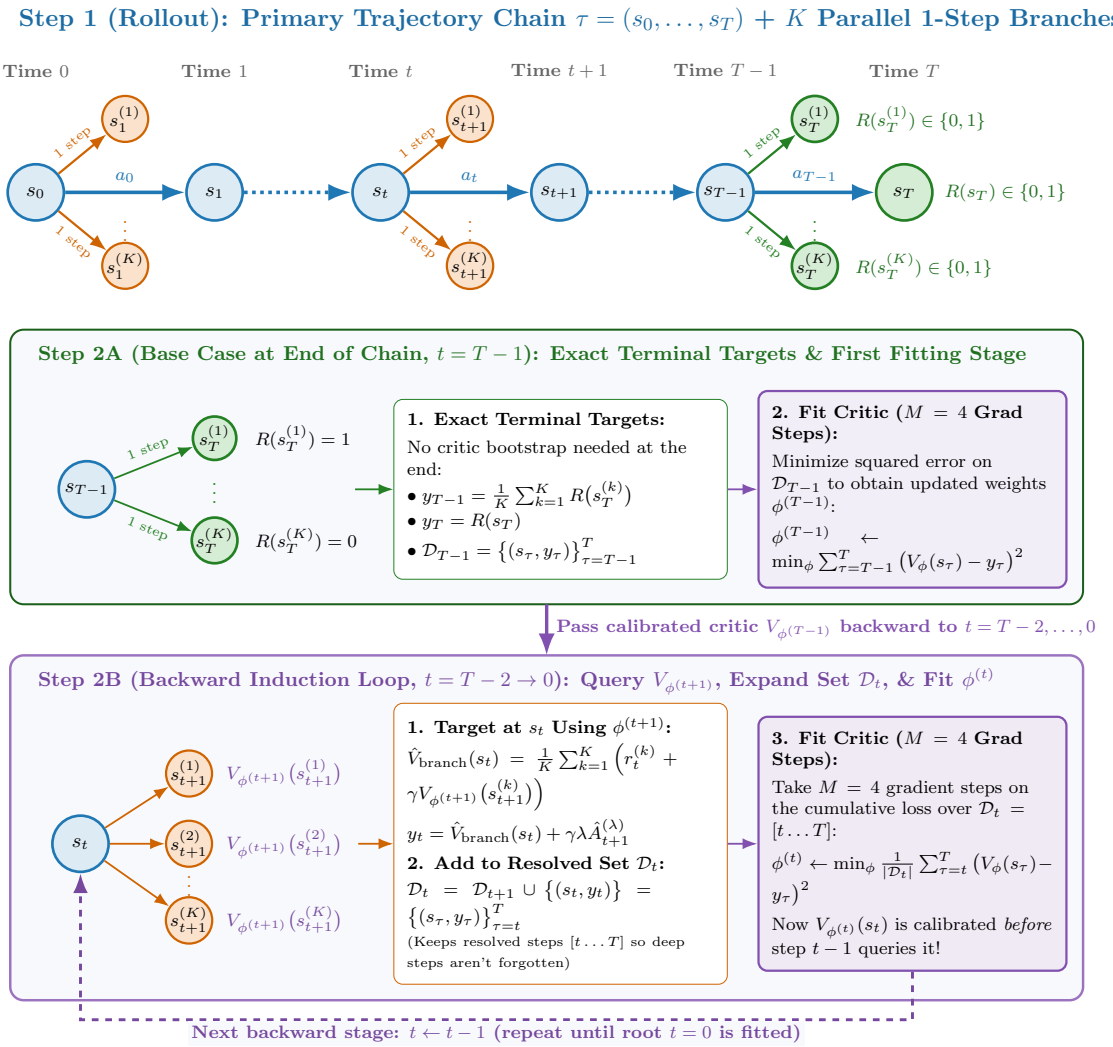


Figure 2: **Algorithm 4: Staged FBI ($\mathbf{FBI_{staged}}(\lambda)$). Top (Step 1):** Roll out the primary chain $\tau = (s_0, \ldots, s_T)$ and fork K parallel 1-step branches $s_{t+1}^{(1..K)}$ from each visited state s_t. **Middle (Step 2A):** At the end of the chain ($t = T-1$), the K 1-step branches land on terminal states $s_T^{(1..K)}$ with exact rewards $R(s_T^{(k)})$, and the critic is trained for $M = 4$ gradient steps to obtain $\phi^{(T-1)}$. **Bottom (Step 2B):** Sweeping backward for $t = T-2$ down to 0, step t queries the *just-updated* critic $V_{\phi^{(t+1)}}$ on its K 1-step children, adds (s_t, y_t) to the resolved set $\mathcal{D}_t = \{(s_\tau, y_\tau)\}_{\tau=t}^{T}$, and takes $M = 4$ gradient steps to produce $\phi^{(t)}$ before stepping backward to $t-1$.